

Projet: un moteur d'ingestion et de traitement de métriques système/IoT en temps réel.

Ce projet combine :

- **POO & Typage statique** (Dataclasses, Protocole/Duck typing, Modèles de données).
- **Décorateurs avancés** (Mesure de temps, validation, gestion du cache/retry).
- **Programmation asynchrone (**asyncio**)** (Producteurs/Consommateurs concurrents, file d'attente non-bloquante).
- **Gestionnaires de contexte** (Gestion propre des sessions/fichiers log).

Architecture du projet

```
moteur_ingestion/  
├── main.py           # Point d'entrée & Orchestration Async  
├── decorateurs.py    # Décorateurs de métriques & logs  
├── modeles.py        # Modèles de données typés & Protocole  
└── pipeline.py       # Proposeur, File d'attente (Queue) & Consommateur
```

1. **modeles.py** : Typage, Dataclasses & Protocoles

Nous définissons le contrat d'interface avec un **Protocol** (Duck typing statique) et notre modèle de métrique.

```
from dataclasses import dataclass, field  
from datetime import datetime  
from enum import Enum, auto  
from typing import Protocol, runtime_checkable  
  
class NiveauAlerte(Enum):  
    INFO = auto()  
    AVERTISSEMENT = auto()  
    CRITIQUE = auto()  
  
@dataclass(frozen=True)  
def Metrique:  
    source_id: str  
    valeur: float  
    unite: str  
    horodatage: datetime = field(default_factory=datetime.now)  
  
@runtime_checkable  
class ProcesseurMetrrique(Protocol):  
    """Contrat d'interface pour les processeurs de métriques."""
```

```
async def traiter(self, metrique: Metrique) -> NiveauAlerte:
    ...
```

2. `decorateurs.py` : Décorateurs Asynchrones

Des décorateurs réutilisables qui fonctionnent sur des coroutines (`async def`).

```
import functools
import logging
import time
from typing import Any, Callable, Coroutine

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(message)s")

def log_execution_async(
    func: Callable[..., Coroutine[Any, Any, Any]]
) -> Callable[..., Coroutine[Any, Any, Any]]:
    """Décorateur pour mesurer la durée et logger l'exécution d'une
    coroutine."""

    @functools.wraps(func)
    async def wrapper(*args: Any, **kwargs: Any) -> Any:
        debut = time.perf_counter()
        resultat = await func(*args, **kwargs)
        duree = time.perf_counter() - debut
        logging.info(
            f"[{func.__name__}] Exécuté en {duree:.4f}s - Résultat:
{resultat}"
        )
        return resultat

    return wrapper
```

3. `pipeline.py` : Traitement Asynchrone & POO

On implémente le processeur de métriques et le pipeline basé sur `asyncio.Queue`.

```
import asyncio
from modeles import Metrique, NiveauAlerte, ProcesseurMetrique
from decorateurs import log_execution_async

class AnalysteTemperature(ProcesseurMetrique):
```

```

"""Implémente le protocole ProcesseurMetrique."""

def __init__(self, seuil_critique: float = 80.0):
    self.seuil_critique = seuil_critique

@log_execution_async
async def traiter(self, metrique: Metrique) -> NiveauAlerte:
    # Simulation d'un traitement I/O lourd (ex: écriture en base)
    await asyncio.sleep(0.1)

    if metrique.valeur >= self.seuil_critique:
        return NiveauAlerte.CRITIQUE
    elif metrique.valeur >= self.seuil_critique * 0.8:
        return NiveauAlerte.AVERTISSEMENT
    return NiveauAlerte.INFO

class MoteurPipeline:
    """Orchestre les producteurs et les consommateurs asynchrones via une
    Queue."""

    def __init__(self, processeur: ProcesseurMetrique):
        self.queue: asyncio.Queue[Metrique] = asyncio.Queue(maxsize=100)
        self.processeur = processeur

    async def produire(self, capteur_id: str, nombre_mesures: int) -> None:
        """Simule un capteur IoT qui pousse des métriques dans la file."""
        import random

        for _ in range(nombre_mesures):
            valeur = random.uniform(20.0, 95.0)
            m = Metrique(source_id=capteur_id, valeur=valeur, unite="°C")
            await self.queue.put(m)
            await asyncio.sleep(0.05) # Fréquence de génération des
données

    async def consommer(self, id_worker: int) -> None:
        """Worker qui dépile et traite les métriques."""
        while True:
            metrique = await self.queue.get()
            try:
                alerte = await self.processeur.traiter(metrique)
                if alerte == NiveauAlerte.CRITIQUE:
                    print(
                        f"🚨 [WORKER {id_worker}] ALERTE CRITIQUE :
{metrique.source_id} = {metrique.valeur:.1f}{metrique.unite}"
                    )
            finally:
                self.queue.task_done()

```

4. main.py : Orchestration Générale

Lancement parallèle de plusieurs capteurs (producteurs) et plusieurs workers (consommateurs).

```
import asyncio
import time
from pipeline import AnalysteTemperature, MoteurPipeline

async def main() -> None:
    processeur = AnalysteTemperature(seuil_critique=75.0)
    pipeline = MoteurPipeline(processeur)

    # 1. Lancement des workers consommateurs en arrière-plan
    workers = [
        asyncio.create_task(pipeline.consommer(id_worker=i + 1))
        for i in range(3) # 3 workers en parallèle
    ]

    # 2. Lancement des capteurs (producteurs) en parallèle
    debut = time.perf_counter()
    await asyncio.gather(
        pipeline.produire("Capteur-Serveur-01", 5),
        pipeline.produire("Capteur-Serveur-02", 5),
        pipeline.produire("Capteur-ChambreFroide", 5),
    )

    # 3. Attente du traitement complet de la file d'attente
    await pipeline.queue.join()

    # 4. Annulation propre des workers permanents
    for w in workers:
        w.cancel()

    fin = time.perf_counter()
    print(
        f"\n✅ Ingestion de 15 métriques terminée en {fin - debut:.2f} secondes !"
    )

if __name__ == "__main__":
    asyncio.run(main())
```